

Porovnání: BigQuery procedura vs. Python + Task Scheduler

Ano, šlo by to udělat i v Pythonu přes .bat a Windows Task Scheduler. Technicky by Python jen dělal to samé, co teď dělá procedura:

1. připojit se do BigQuery
2. vylistovat kandidátní tabulky z INFORMATION_SCHEMA
3. vyfiltrovat __dq10000__, starší než 12 hodin, total_rows > 0
4. složit UNION ALL / INSERT SQL
5. spustit query
6. případně později dropnout zdrojové tabulky
7. zalogovat výsledek

Za mě ale pro tenhle konkrétní případ je lepší procedura přímo v BigQuery.

Varianta A: BigQuery procedura + Scheduled Query

To je to, co teď stavíme.

Výhody:

- běží přímo v BigQuery, tedy blízko datům
- není potřeba server, Python, .bat ani Task Scheduler
- méně závislostí na Windows prostředí
- jednodušší oprávnění v GCP
- logy běhů jsou v BigQuery / Scheduled Queries
- dobře se hodí na čistě SQL operaci
- UNION ALL a INSERT se provede přímo v BQ enginu

Nevýhody:

- horší komfort pro složitější programování než Python
- dynamické SQL v proceduře je méně přehledné
- ladění je trochu méně pohodlné
- pokud chceme pokročilé logování, musíme si udělat auditní tabulky
- práce s výjimkami je omezenější než v Pythonu

Pro náš případ ale výhody převažují, protože proces je hlavně SQL operace:

metadata -> UNION ALL -> INSERT -> později DROP

Varianta B: Python + .bat + Windows Task Scheduler

Taky by to šlo. Python skript by se spouštěl například z NTINFO serveru přes .bat.

Příklad principu:

```
@echo off
cd /d F:\dp_arch\code\dq10000_collect
python collect_dq10000.py
```

A Python by používal knihovnu:

```
google-cloud-bigquery
```

Výhody Pythonu:

- lepší logování do souboru
- jednodušší složitější podmínky a větvení
- lepší práce s retry logikou
- lepší ošetření chyb
- můžeš snadno posílat e-mail/notifikaci
- můžeš mít konfiguraci v YAML/JSON
- vhodnější, pokud se bude proces rozrůstat mimo samotné SQL

Nevýhody Pythonu:

- musí běžet nějaký server
- musí být správně nainstalovaný Python a knihovny
- musí být vyřešené přihlášení do GCP
- riziko problémů s ADC / service account / proxy / certifikáty
- Windows Task Scheduler je další provozní závislost
- pokud server neběží, neběží ani proces
- horší předání do čistého GCP provozu

U tebe navíc víme, že v O2 prostředí už řešíte servery, proxy, GCloud SDK, ADC, service accounty a Windows/PROD scheduler. Takže Python varianta by byla možná, ale přidává další věci, které se můžou rozbít.

Moje doporučení pro tento konkrétní use-case

Pro aktuální řešení bych zvolil:

BigQuery procedura + Scheduled Query

Důvod:

Celá logika je uvnitř BigQuery a pracuje jen s BigQuery tabulkami.

Není potřeba tahat data ven, není potřeba Python a není potřeba Windows server.

Python bych volil až ve chvíli, kdy by se požadavky změnilly například na:

- posílat e-mailové reporty
- číst soubory z disku nebo GCS
- dělat složitější kontrolní logiku

- volat externí API
- řídit více systémů najednou
- dělat pokročilé retry/backoff
- integrovat s interním oflow mimo BigQuery

Praktické srovnání

Kritérium	BQ procedura	Python + Task Scheduler
Čisté SQL zpracování	velmi vhodné	možné, ale zbytečně těžší
Provoz bez serveru	ano	ne
Přenos dat mimo BQ	ne	ne, pokud se dobře napíše
Logování	přes BQ joby + audit tabulky	velmi flexibilní
Ladění	horší	lepší
Oprávnění	řeší se v GCP	řeší se GCP + Windows
Stabilita provozu	lepší	závisí na serveru
Vhodné pro náš DQ10000 proces	ano	spíš alternativa

Doporučený další postup

Za mě bychom pokračovali takto:

1. Nechat logiku v BQ proceduře.
2. Přidat auditní tabulku běhů.
3. Přidat ochranu proti duplicitám.
4. Až pak přidat DROP zdrojových tabulek.
5. Python si nechat jen jako záložní/provozní variantu, pokud by Scheduled Query nevyhovovala.

Takže odpověď: ano, v Pythonu by to šlo, ale pro tento případ je čistší a provozně lepší BigQuery procedura přímo v BQ.